

xui-skia

Skia 渲染后端 · 代码流程详解

本文档基于 `xui-skia` 当前分支 `ani` 的源码逐行分析而成，覆盖从 `App::render` 进入后端、到像素出现在窗口上的完整调用链：帧生命周期的三个阶段、损伤（damage）计算与增量重绘、图层渲染计划的执行、四类缓存与资源池、文本与字形通路，以及四种平台呈现后端。

项	内容
crate	<code>xui-skia</code> v0.1.0 · edition 2024
核心依赖	<code>skia-safe 0.99</code> (gpu + textlayout) 、 <code>softbuffer</code> 、 <code>winit 0.30</code> 、 <code>moka</code> 、 <code>memmap2</code>
内部依赖	<code>xui</code> (帧/场景契约) 、 <code>xui-interface</code> (共享类型) 、 <code>xui-render-graph</code> (图层计划 IR)
实现的契约	<code>xui::render::RenderBackend<TextHost<T>></code> 与 <code>xui_interface::TextBackend</code>
平台呈现	macOS = Metal · Windows = Direct3D 12 · Linux = Vulkan · 全平台兜底 = <code>softbuffer</code> CPU blit
源码规模	8 个平台无关模块 + 3 个平台模块，约 6,500 行实现 + 570 行集成测试

目录

1 定位与模块地图	2 输入契约：BuiltFrame
3 帧生命周期总览	4 表面模型：GPU 双表面与软件单表面
5 阶段一：begin_frame	6 阶段二：submit —— 绘制主流程
7 图层实例的执行：渲染计划	8 阶段三：end_frame —— 呈现
9 损伤跟踪：DamageTracker	10 缓存与资源池体系
11 文本与字体流程	12 平台呈现后端
13 效果实现与 SkSL	14 错误语义、优化开关与统计
15 测试与调试	16 代码位置索引

1 定位与模块地图

xui 的渲染路径被切成三段，后端只做最后一段。`xui::render::scene` 维护一棵保留式、与后端无关的场景树；`xui-render-graph` 把静态样式编译成可复用的 `LayerProgram`，再按当帧几何实例化为可执行的 `LayerRenderPlan`（纯 Rust，无 GPU 依赖，`#![forbid(unsafe_code)]`）；`xui-skia` 消费这两者，光栅化到窗口表面。核心运行时从不直接命名任何 GPU 库。

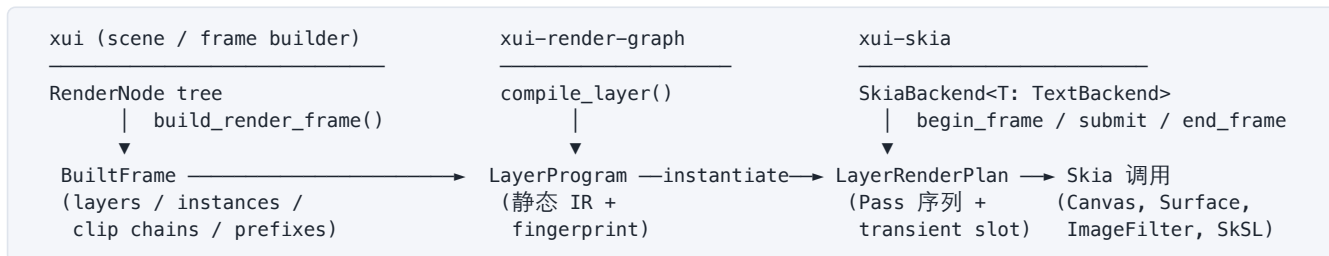


图 1 三段式渲染路径中 xui-skia 的位置

1.1 模块划分

模块	行数	职责
<code>backend.rs</code>	4485	核心。 <code>SkiaBackend</code> 结构体、 <code>RenderBackend</code> 实现、帧分析、图层遍历、渲染计划执行、图元绘制（形状/矢量/图像/文本）、SkSL 常量、几何与颜色转换工具
<code>damage.rs</code>	727	<code>DamageTracker</code> 与 <code>DamageRegion</code> 。逐层快照 diff、跨层损伤传播、区域简化（合并/分带）
<code>cache.rs</code>	195	<code>LayerSurfaceCache</code> 。按 <code>LayerCacheId</code> 复用图层离屏表面，含预算驱逐与命中统计
<code>present.rs</code>	198	<code>WindowPresenter</code> 枚举。运行时挑选 GPU 呈现器，失败回落 softbuffer
<code>text.rs</code>	708	<code>SkiaTextBackend</code> 。实现 <code>FontDatabase</code> / <code>Shaper</code> / <code>GlyphRasterizer</code> ，基于 <code>SkParagraph</code>
<code>metal.rs</code>	135	macOS： <code>CAMetalLayer</code> + <code>MTLCommandQueue</code> + Ganesh Metal 上下文
<code>d3d.rs</code>	335	Windows：D3D12 设备/队列 + DXGI flip-model 交换链 + 单 fence 帧节奏
<code>vulkan.rs</code>	626	Linux： <code>VkInstance/VkDevice</code> + <code>VK_KHR_swapchain</code> ，CPU 同步
<code>error.rs</code> / <code>stats.rs</code>	47 / 30	<code>SkiaBackendError</code> 全部错误分类； <code>SkiaFrameStats</code> 逐帧计数器

1.2 公开 API 面

```

pub use backend::{SkiaBackend, SkiaBackendOptions, SkiaOptimizations};
pub use cache::SkiaLayerCacheStats;      pub use error::SkiaBackendError;
pub use stats::SkiaFrameStats;
pub use text::{SkiaFontId, SkiaGlyphKey, SkiaParagraphState, SkiaTextBackend};
  
```

应用不直接构造后端：`xui-winit` 的 `skia` 特性（默认开启）在 `runner()` 里调用 `SkiaBackend::new(window)` 并把它交给 `WinitRunner`。`SkiaBackend::headless(scale, options)` 提供无窗口的纯软件路径，测试与基准用它。

2 输入契约：BuiltFrame

后端每帧收到的唯一数据是一个 `&BuiltFrame`（外加可变的 `TextHost<T>`）。理解它的字段是理解整个绘制流程的前提。

```

pub struct BuiltFrame {
    pub root_layer: BuiltLayerId,           // 根图层索引
    pub layers: Vec<BuiltLayer>,           // 扁平化的图层数组（父先于子入栈）
    pub layer_instances: Vec<BuiltLayerInstance>,
    pub composite_prefixes: Vec<CompositePrefix>, // backdrop 读取点的“目的地前缀”
    pub clip_chains: Vec<BuiltClipChain>,     // 子→父链式裁剪
    pub live_layer_caches: Vec<LayerCacheId>, // 本帧仍存活的缓存槽
    pub scene_revision: u64, pub properties_revision: u64,
}

pub struct BuiltLayer {
    source: RenderNodeId,
    content_bounds: Bounds,
    render_bounds: Bounds, // 含效果外扩
    content_version: ContentVersion,
    cache_id: Option<LayerCacheId>,
    cache_policy: CachePolicy,
    backdrop_isolation: BackdropIsolation,
    items: Vec<BuiltItem>, // Draw | Layer
}

pub struct BuiltLayerInstance {
    source: RenderNodeId,
    layer: BuiltLayerId, // 指向被放置的子层
    composite: CompositeInstance, // 动态值（不入指纹）
    render_program: BuiltLayerProgram, // 静态 IR + 掩码
    clip_chain: Option<BuiltClipChainId>,
    world_bounds: Bounds,
    placement_version: PlacementVersion,
    destination_prefix: Option<CompositePrefixId>,
}

```

概念	含义与后端如何使用
BuiltItem	图层内的有序条目，只有两种： Draw(BuiltDraw) (Shape / Vector / Image / Text 四类图元) 与 Layer(BuiltLayerInstanceId) (子图层的一次放置)。顺序即绘制顺序，也是 backdrop 前缀的定位依据。
content_bounds vs render_bounds	前者是内容自身范围，后者已包含模糊/阴影等效果的视觉外扩。离屏表面按 render_bounds 分配，掩码与 backdrop-only 通路用 content_bounds 。
destination_prefix	只有声明了显式目的地读取 (backdrop) 的实例才有。它记录“该操作开始前观察到的逻辑目的地”，后端在 draw_layer 里严格校验它必须等于当前所站的 (图层, 已经条目数)。
backdrop_isolation	Passthrough 表示子层不隔离 backdrop，父层的背景要透传进去；这是 analyze_frame 需要沿边传播的原因。
cache_policy / cache_id	CachePolicy::None 以外的图层才有缓存身份。 LayerSurfaceCache 按 cache_id 存表面， live_layer_caches 决定帧末保留哪些。
content_version / placement_version	损伤跟踪的比较依据：内容版本变了说明画的东西变了，放置版本变了说明位置/变换变了。

3 帧生命周期总览

上层驱动来自 [xui/src/app.rs:224 App::render](#)，它是后端唯一的入口序列：

```

App::render(backend, text)
├─ drain_async_messages()           // 异步消息入队
├─ rebuild_sync_if_needed()         // 首帧同步构建；后续帧走分片 rebuild_slice_if_needed()
│   └─ flush_node_lifecycle(backend, text) → backend.handle_node_lifecycle(&event)
├─ ui_runtime.update_tree(size, text) // 布局 + 文本测量
├─ ui_runtime.build_render_frame()?   // → Option<Frame{ built: BuiltFrame }>
│   └─ None ⇒ 本帧无事可做，直接返回 Ok(())
├─ backend.begin_frame(self.size)?    // 1. 准备
├─ backend.submit(&frame.built, text)? // 2. 绘制
├─ backend.end_frame()?               // 3. 呈现
└─ if backend.did_present() { ui_runtime.finish_render_frame(&frame) } // 提交版本号

```

图 2 上层调用序列（三阶段是硬契约：submit 前必须 begin_frame，end_frame 前必须 submit）

为什么 did_present 很重要

后端在软件路径上可能因为损伤区域为空而跳过整个呈现步骤。只有真正呈现了，上层才会 [finish_render_frame](#) 推进版本号；否则下一帧的 diff 仍以旧版本为基准，已经绘制但没上屏的内容不会被误判为“已交付”。

3.1 后端内部的三阶段

```

┌────────────────── begin_frame(size) ───────────────────┐
| ensure_surface(logical)      逻辑尺寸 × scale_factor → 物理像素；必要时重建交换链/表面
| frame_index += 1 ; frame_stats = default()
| 每 120 帧修剪一次 text_blob_cache (丢弃 120 帧未用的条目)
| GPU 路径: presenter.acquire_surface(ctx, w, h) → self.raster = 交换链图像
| surface_pool.begin_frame() ; submitted = presented = false
└──────────────────┘

┌────────────────── submit(frame, text) ───────────────────┐
| 1.root_damage = damage_tracker.update(frame)      ← 全帧 diff, 见 §9
| 2.layer_cache.begin_frame(dirty_region_count)
| 3.surface = if on_gpu { compositor.take() } else { raster.take() }
| 4.draw_frame(&mut surface, frame, &root_damage, text)  ← 绘制主流程, 见 §6
| 5.on_gpu ⇒ blit_to_swapchain(&mut surface)      整帧一次纹理四边形拷贝到交换链图像
| 6.归还 surface; 失败则 damage_tracker/layer_cache/surface_pool 全清并返回 Err
| 7.layer_cache.finish_frame(live_layer_caches, budget) ; pending_damage = root_damage
└──────────────────┘

┌────────────────── end_frame() ───────────────────┐
| !submitted ⇒ Err(InvalidFrame)
| GPU 路径: presenter.present(ctx, raster.take()) → flush + submit + 平台 Present
| 软件路径: 按 buffer_age 补历史损伤 → copy_surface_damage → present_with_damage
| presented = true
└──────────────────┘

```

图 3 三阶段职责切分

4 表面模型：GPU 双表面与软件单表面

这是整个后端最关键的一个设计决策，也是增量重绘能在 GPU 上成立的原因。交换链图像每帧取回来时内容是未定义的——直接往里画就等于每帧全量重绘，损伤跟踪算出来的一切都白费。因此 GPU 路径引入了第二块常驻表面。

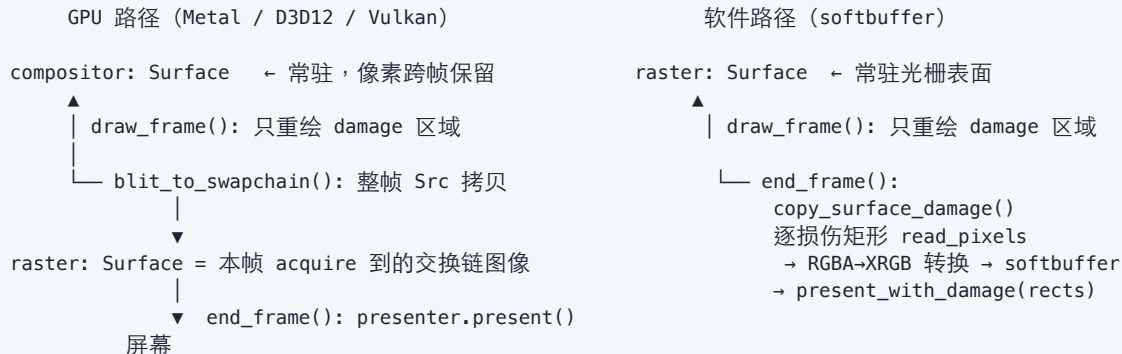


图 4 两条路径的表面拓扑

- 全帧 blit 的代价是一个屏幕大小的带纹理四边形；全帧重绘的代价是整个场景。前者恒定且极低，所以“画到常驻表面 + 整帧 blit”严格优于“直接画到交换链图像”。
- blit_surface 用 BlendMode::Src 而非 SrcOver：目的地是内容未定义的交换链图像，必须整体替换而不是混合。它还会 reset_matrix() + save/restore，所以画布上遗留的任何变换都不会影响这次一比一拷贝（blit_ignores_the_canvas_transform 覆盖这点）。
- blit_to_swapchain 前先 context.flush_and_submit_surface(source, None)：呈现器只 flush 交换链表面，场景的绘制工作必须显式对随后的读取可见。

4.1 ensure_surface 的分支

```
fn ensure_surface(logical: Size<f32>):
    w = ceil(logical.width * scale).max(1)      h = ceil(logical.height * scale).max(1)

    if gpu_context.is_some():
        if frame_size_px != (w,h):
            frame_size_px = (w,h)
            raster = None ; compositor = None      // 重建交换链会释放其图像，不能有任何残留引用
            damage_tracker.clear() ; layer_cache.clear() ; surface_pool.clear() ; damage_history.clear()
            presenter.resize(gpu_context, w, h)?    // ← 这里必须传 ctx，见 §12
        if compositor.is_none():
            compositor = new_surface_px(w,h,ctx)?    // 全新表面 ⇒ 下一帧必须全量重绘
            damage_tracker.clear()
        return Ok(())

    // 软件路径
    if frame_size_px != (w,h) || raster.is_none():
        raster = surfaces::raster_n32_premul((w,h)) // None ⇒ Err(SurfaceAllocation)
        frame_size_px = (w,h) ; 四个缓存/历史全清
```

尺寸或 DPI 变化时把损伤跟踪器、图层缓存、表面池、损伤历史一次性清空，是“任何跨帧假设失效就全部作废”这一原则的体现。`set_factor` 走的是同一套逻辑（外加把两块表面都置 `None`），`invalidate()` 则是它的公开版本——唯独保留 `compositor`，因为重新分配一块屏幕大小的表面比它想省下的那次重绘还贵。

5 阶段一：begin_frame

职责很窄：把表面准备好、把逐帧状态归零、在 GPU 路径上取到本帧的交换链图像。

步骤	说明
<code>ensure_surface(size)</code>	见 §4.1。逻辑尺寸转物理像素，必要时重建。
<code>frame_index</code> 自增	<code>wrapping_add(1).max(1)</code> ——0 被保留为“从未使用”的哨兵值。
<code>frame_stats</code> 重置	只保留 <code>frame_index</code> ，其余计数器清零。所有统计都是逐帧的。
文本 blob 修剪	每 120 帧一次：丢弃 <code>last_used_frame < frame_index - 120</code> 的条目。摊销式清理，避免每帧遍历整张缓存表。
获取交换链图像	仅 GPU 路径： <code>presenter.acquire_surface(ctx, w, h) → self.raster</code> 。Metal 取 <code>nextDrawable</code> ，D3D12 取当前后台缓冲并等 fence，Vulkan 取 <code>acquireNextImage</code> 。
<code>surface_pool.begin_frame()</code>	推进池的帧号并驱逐闲置超过 <code>MAX_POOL_IDLE_FRAMES = 60</code> 帧的尺寸档。
标志位复位	<code>submitted = presented = damage_rollback_pending = false</code>

6 阶段二：submit —— 绘制主流程

6.1 入口逻辑

第一件事是 `damage_tracker.update(frame)`，它返回根图层的损伤区域并顺带把每层的损伤存进 `last_damage`（后续子层重绘要查）。注释说明了一个刻意的取舍：跟踪器不做“先克隆再回滚”，因为那等于每帧复制一份完整场景快照；失败时直接 `clear()` 走全量重绘——目标表面已经被写了一半，唯一可靠的恢复就是全部重画。

```
fn submit(frame, text):
    root_damage = damage_tracker.update(frame)          // $9
    if *DEBUG_FRAME { eprintln!("[frame N] ... root_damage_rects=.. area=.. gpu=..") }
    frame_stats.root_damage_rects / root_damage_area_sum = ...
    layer_cache.begin_frame(damage_tracker.dirty_region_count())

    on_gpu = gpu_context.is_some()
    surface = if on_gpu { compositor.take() } else { raster.take() }
               .ok_or(InvalidFrame("begin_frame must be called before submit"))?

    result = draw_frame(&mut surface, frame, &root_damage, text)
    if result.is_ok() && on_gpu { result = blit_to_swapchain(&mut surface) }
    归还 surface (无论成败)

    if let Err(e) = result {                               // 失败：放弃一切增量假设
        damage_tracker.clear(); layer_cache.clear(); surface_pool.clear(); return Err(e)
    }
    layer_cache.finish_frame(&frame.live_layer_caches, options.layer_cache_budget_bytes)
    damage_rollback_pending = true; pending_damage = root_damage; submitted = true
```

6.2 analyze_frame：单次遍历回答两个问题

这一遍走过每个图层的每个条目，替代了历史上三次独立遍历（校验 backdrop 前缀、上传图像数据、判断哪些图层含有读取目的地的东西）。它做两件事：

- 上传图像：遇到 `BuiltDraw::Image` 就调 `prepare_image`，按 `ImageKey` 查 `source_images` 缓存，`data_id` 不匹配才重新 `make_image` 并插入。
- 计算 `needs_backdrop`：若实例的程序声明了 `ExternalResourceKind::Backdrop` 外部资源，则其所在图层置位；若子层是 `BackdropIsolation::Passthrough`，记一条 `(parent, child)` 传播边。

```
// 传播：需求沿 passthrough 边从子传到父
let mut changed = !edges.is_empty();
while changed {
    changed = false;
    for &(parent, child) in edges.iter().rev() {           // 子层总在父层之后入栈 = 逆序一遍通常就收敛
        if needs_backdrop[child] && !needs_backdrop[parent] { needs_backdrop[parent] = true; changed = true }
    }
}
```

循环形式而非单遍：正确性不依赖入栈顺序，且每轮要么置位要么停止，必然终止。注意它不做前缀校验

——`draw_layer` 走到实例时会做更严格的检查（还要求前缀恰好指向当前所站的条目位置），中途失败由 `submit` 的全量重绘兜底。

6.3 draw_frame → redraw_layer_region

```
fn draw_frame(surface, frame, damage, text):
    analysis = analyze_frame(frame)?
    viewport = Bounds::from_zero_size(frame_size_px.to_f32() / scale_factor)
    if !damage.is_empty():
        redraw_layer_region(surface, viewport, frame, frame.root_layer,
                             None /*inherited_backdrop*/, damage, options.clear_color, text, &analysis)?

fn redraw_layer_region(target, target_bounds, frame, layer_id, inherited_backdrop, damage, clear_color, ..):
    region = damage_region(target, target_bounds, scale_factor, damage) // Bounds → 物理像素 Skia Region
    if region.is_empty() { return Ok(()) }
    canvas.save(); canvas.reset_matrix(); canvas.clip_region(&region, None); canvas.clear(clear_color)
    draw_layer(target, target_bounds, frame, layer_id, inherited_backdrop, damage, text, analysis)?
    canvas.restore_to_count(save)
```

根层用 `options.clear_color` 清底（默认 `Color::rgba(0.08,0.09,0.11,1.0)`），子层离屏表面用

`Color::TRANSPARENT`。清屏被裁剪在损伤区域内，所以未损伤的像素在常驻表面上保持不变——这正是增量重绘生效的位置。`damage_region` 在转换时对每个矩形做 `floor/ceil` 外扩并夹到表面边界，保证物理像素覆盖不小于逻辑损伤。

6.4 draw_layer：逐条目绘制

这是整棵树的递归主体。对每个 `BuiltItem` 分两路：

```
for (item_index, item) in layer.items.iter().enumerate():

  BuiltItem::Draw(draw)
  if cull && !damage.intersects(common.world_bounds) { items_culled += 1 ; continue }
  canvas.save()
  configure_canvas(canvas, target_bounds, scale) // scale + translate(-bounds.origin)
  apply_clip_chain(canvas, frame, common.clip_chain, IDENTITY)? // 递归：外层先裁
  match draw {
    Shape => draw_shape(canvas, &mut gradients, prim, world_transform, 1.0)
    Vector => compiled_vector_scene(scene) → draw_vector(...)
    Image => draw_image(canvas, prim, world_transform, 1.0)?
    Text  => draw_text(canvas, prim, world_transform, 1.0, text)?
  }
  canvas.restore_to_count(save)

  BuiltItem::Layer(instance_id)
  1.backdrop 前缀严格校验（若程序声明了 Backdrop 外部资源）
    prefix.local.layer == layer_id && prefix.local.item_count == item_index 否则 Err
  2.plain = !program_needs_backdrop && !child_needs_backdrop
    visible_bounds = program.layer_visual_expansion().apply_to_bounds(instance.world_bounds)
    if cull && plain && !damage.intersects(visible_bounds) { layer_instances_culled += 1 }
  3.物化 backdrop（需要时）：snapshot_target(target) [+ 与继承 backdrop 合成]
  4.passthrough 子层：execute_backdrop_only(...) → 逆变换回子层坐标 = child_backdrop
  5.取子层表面：layer_cache.acquire(child_layer, scale) → Hit(lease) | Miss{w,h}
  6.child_damage = if lease.reused { damage_tracker.layer(src) } else { full(child_bounds) }
  7.if !child_damage.is_empty() { redraw_layer_region(lease.surface, child_bounds, ...) }
  8.child_image = snapshot_target(lease.surface) ; spent = layer_cache.release(...)
  9.execute_instance(target, .., &child_image, backdrop|child_image, Complete)? ← $7
  10.drop(child_image) ; if let Some(s) = spent { recycle_surface(s) }
```

图 5 draw_layer 的两条分支

剔除为什么是安全的

剔除的正确性完全建立在“损伤跟踪覆盖了每一处变化”这个前提上——而这正是缓存图层路径本来就必须满足的契约。Skia 自己也会拒绝落在裁剪外的绘制，但那是在 save、裁剪链重建、paint 构造、draw 调用都付过账之后。提前一次包围盒相交测试把这些全省了。`tests/optimization_parity.rs` 用像素比对守住这条不变量。

6.5 backdrop 的三种情形

情形	行为
都不需要	<code>backdrop_materializations_avoided += 1</code> , <code>backdrop = None</code> 。此时 <code>execute_instance</code> 用 <code>child_image</code> 顶替 backdrop 参数（计划不会读它）。
实例自身读目的地	对目标表面 <code>image_snapshot()</code> 得到前缀图像；若有继承 backdrop，先 <code>composite_images(inherited, prefix)</code> 叠一层。
子层 passthrough 且内部有读目的地者	额外走 <code>execute_backdrop_only</code> ：新开表面画上 backdrop，用 <code>LayerProgramEntry::BackdropOnly</code> 只跑程序的 backdrop 分支（层内容用 1×1 透明像素顶替），再用 <code>inverse_affine(composite.transform)</code> 把结果变换回子层坐标系传下去。

7 图层实例的执行：渲染计划

7.1 实例化

`execute_instance` 把当帧几何塞进 `LayerPlanContext`，让 `xui-render-graph` 把静态程序降级为一串可执行的 Pass：

```
let plan = instance.render_program.program().instantiate_entry(entry, &LayerPlanContext {
    backdrop_source_bounds:    backdrop.bounds,
    parent_destination_bounds: target_bounds,
    composite_clip_bounds:     target_bounds & instance.world_bounds,
    layer_content_bounds:      child.content_bounds,
    backdrop_bounds:           target_bounds & instance.world_bounds,
    composite:                  instance.composite,      // 动态值：不透明度/变换/混合
    scale_factor,
    color_texture_class:        TextureClass::LINEAR_COLOR,
    external_aliasing:          ExternalAliasing::Distinct,
    limits:                     PlanLimits::default(),
});
```

`entry` 是 `LayerProgramEntry::Complete`（完整通路）或 `BackdropOnly`（只跑 backdrop 分支，§6.5）。实例化失败以 `PlanError` 的形式经 `#[from]` 变成 `SkiaBackendError::RenderPlan`。

7.2 execute_plan：资源分配与逐 pass 执行

```
fn execute_plan(target, target_bounds, frame, instance, plan, layer_content, backdrop):
    统计: render_plans, render_passes, planned_transient_{resources,slots,texels}, peak_live_texels,
          transient_surface_allocations = plan.slots().len(),
          transient_surface_reuses      = transient_resource_count - transient_slot_count

    // 1.按计划分配的“槽”一次性开表面（槽 = 计划做过生命周期分析后的复用单位）
    transient_surfaces = plan.slots().map(|s| new_surface_px(s.extent.w, s.extent.h?))
    values: Vec<Option<RasterImage>> = vec![None; plan.resources().len()]

    // 2.顺序执行
    for (pass_index, pass) in plan.passes().enumerate():
        if pass.output == plan.parent_destination():
            execute_composite_pass(...) // 直接画进父目的地，见 7.4
        else:
            slot = plan.resources()[pass.output.index()].slot.ok_or(InvalidFrame(..))?
            bounds = plan_resource_bounds(plan, pass.output, scale_factor)
            values[pass.output.index()] = Some(execute_filter_pass(.., &mut transient_surfaces[slot]))?

    // 3.生命周期回收：最后一次读（或产出）就发生在本 pass 的资源，立刻丢引用
    for (resource, value) in plan.resources().iter().zip(&mut values):
        if resource.last_read.or(resource.producer) == Some(pass_index) { *value = None }

    drop(values) // 先丢快照，再还表面
    for surface in transient_surfaces { recycle_surface(surface) }
```

注意末尾的顺序：`values` 里的 `Image` 是对这些表面的快照，必须先全部释放再把表面放回池子——否则下一次往带有活快照的表面上画，Skia 会做写时复制，池化省下的分配又被拷贝吃掉。`SurfacePool` 的注释与 `a_recycled_surface_carries_no_canvas_state` 测试都在守这条。

7.3 execute_filter_pass : 把 PassOp 映射到 Skia

PassOp	Skia 实现
Copy	无 filter，直接 <code>draw_raster_image</code>
GaussianBlur{axis,sigma_px}	<code>image_filters::blur(($\sigma,0$) (0,σ), TileMode::Decal)</code> ， σ 先除以 <code>scale_factor</code> 回到逻辑单位
ColorMatrix([f32;20])	<code>color_filters::matrix</code> 包成 <code>image_filters::color_filter</code>
ExtractAlpha	常量色矩阵：RGB 归零、A 保留 (<code>extract_alpha_filter()</code>)
AlphaSpread{axis,radius}	<code>image_filters::dilate((r,0) (0,r))</code> ，单轴形态学膨胀
Pixelate{block_w,block_h}	SkSL 运行时着色器 <code>PIXELATE_SKSL</code> ，uniform <code>block</code>
Refraction{strength,chroma}	SkSL <code>REFRACTION_SKSL</code> ，uniform <code>center / amount</code>
ChromaticAberration{offset}	SkSL <code>CHROMATIC_ABERRATION_SKSL</code> ，uniform <code>offset</code>
ShadowComposite{color,offset}	<code>render_shadow</code> ：先用 <code>shadow_color_filter</code> 把 alpha 图按偏移画成阴影，再原图盖上
ApplyMask{transform,mask}	<code>apply_texture_mask</code> ：画源，再用 <code>BlendMode::DstIn</code> 画变换后的掩码
BackdropComposite / LayerComposite	只可能出现在指向父目的地的 pass 上，见 7.4；出现在这里是 <code>unreachable!()</code>

除 `ShadowComposite` 与 `ApplyMask` 外，其余都走统一收尾：

```
fn filter_image(surface, source, bounds, filter):
    clear_surface_output(surface, bounds, scale) // 只清 bounds 对应的物理矩形，池化表面可能更大
    paint.set_anti_alias(true); paint.set_image_filter(filter)
    draw_raster_image(surface.canvas(), bounds, scale, source, Affine::IDENTITY, &paint)
    snapshot_surface_output(surface, bounds) // image_snapshot_with_bounds(0,0,w,h)
```

`snapshot_surface_output` 用带界快照而不是整表面快照，正是因为从池子里拿到的表面尺寸可能大于本次 pass 的输出尺寸（池按精确像素尺寸匹配，但 `clear_surface_output` 与快照都按 `bounds` 算），保证语义只覆盖有效区域。

7.4 execute_composite_pass : 写回父目的地

```
BackdropComposite { opacity, blend_mode, mask, bounds }
source = resolve_resource(pass.inputs[0])
if mask != PlanMask::None:
    mask_image = render_plan_mask(...)
    source = apply_rendered_mask(&source, &mask_image, bounds)
canvas.save()
configure_canvas + clip_rect(bounds)
apply_clip_chain(instance.clip_chain)
paint.alpha = opacity.clamp(0,1)
paint.set_blend_mode(sk_blend_mode(blend_mode))
draw_image_logical(canvas, &source, IDENTITY, &paint)
canvas.restore_to_count(save)

LayerComposite { opacity, transform,
                 blend_mode, operator, bounds }
source = resolve_resource(pass.inputs[0])
canvas.save()
configure_canvas + clip_rect(bounds)
apply_clip_chain(instance.clip_chain)
paint.alpha = opacity.clamp(0,1)
if blend != Normal && op != SrcOver:
    paint.set_blender(runtime_blender(..)) // SkSL
else:
    paint.set_blend_mode(composite_blend_mode(..))
draw_image_logical(canvas, &source, transform, &paint)
```

两者的差别在于：`backdrop` 合成的源已经位于目的地坐标系（用 `IDENTITY`），而图层合成要带上 `composite.transform`。混合模式与合成算子的组合超出 Skia 内建 `BlendMode` 表达能力时（既非 `Normal` 又非 `SrcOver`），退化到 `COMPOSITE_SKSL` 运行时 `blender`，用两个 `int` uniform（`blend_index / operator_index`）在着色器里选分支。

7.5 resolve_resource：计划资源 → 实际图像

```
fn resolve_resource(target, target_bounds, instance, plan, id, values, layer_content, backdrop):
    if let Some(v) = values[id.index()] { return Ok(v) } // 已由前面的 pass 产出
    match plan.resources()[id.index()].kind {
        Transient => Err(MissingResource(idx)) // 该产出而未产出 => 计划或执行有 bug
        External(Backdrop) => backdrop.clone()
        External(ParentDestination) => snapshot_target(target, target_bounds) // 即时快照
        External(LayerContent) => layer_content.clone()
        External(BackdropMask | LayerMask(_)) => resolve_mask_image(instance, kind) // 走 source_images 缓存
    }
```

`resolve_mask_image` 从 `instance.render_program.handle(kind)` 拿到 `ImageResource: Data{key,data}` 会按 `data_id` 校验并按需上传, `Key(key)` 只查缓存, 缺失即 `MissingMaskImage`。返回的 `RasterImage` 的 `bounds` 被置为单位方框——掩码的实际定位由 `PlanMask` 里的变换负责。

8 阶段三：end_frame —— 呈现

前置条件：`submitted` 必须为真，否则 `InvalidFrame`。之后两条路径完全分开。

8.1 GPU 路径

```
let surface = self.raster.take()?; // 本帧 acquire 到的交换链图像（内容已由 blit 填满）
presenter.present(context, surface)?; // 各后端：flush(+access) → submit → 平台 Present
damage_rollback_pending = false; pending_damage = default(); presented = true; return Ok(())
```

表面按值传入：每个后端都必须在把图像交还给呈现引擎之前丢掉自己对它的引用。

8.2 软件路径：buffer age 与历史损伤

softbuffer 的后台缓冲是轮换的，`buffer.age()` 告诉你手上这块缓冲是几帧前用过的。只拷贝本帧损伤是不够的——这块缓冲上残留的是 `age` 帧之前的内容，中间那些帧的损伤也得补上。

```
if presenter.is_some() && !pending_damage.is_empty():
    current_damage = physical_damage_rects(&pending_damage, scale, frame_size_px)
    if current_damage.is_empty() { return Ok(()) } // 亚像素损伤取整后消失 => 无需呈现
    presenter.surface.resize(w, h)?
    let mut buffer = presenter.surface.buffer_mut()?
    let age = usize::from(buffer.age())

    let mut copy_damage = current_damage.clone()
    if age == 0 || age > damage_history.len() + 1 {
        copy_damage = vec![full_softbuffer_rect(frame_size_px)?] // 历史不足 => 整帧拷贝
    } else {
        for historical in damage_history.iter().take(age - 1) { copy_damage.extend(historical) }
    }

    copy_surface_damage(raster, &mut buffer, frame_size_px.width, &copy_damage)?
    buffer.present_with_damage(&current_damage)? // 只提交本帧真正变了的区域
    damage_history.push_front(current_damage); damage_history.truncate(8)
```

`copy_surface_damage` 逐矩形 `read_pixels` 成 `RGBA8888/Unpremul`，再逐像素打包成 softbuffer 要的 `0x00RRGGBB`。这是软件路径最贵的一步，也是 `DamageRegion::simplify` 存在的直接动机——每多一个矩形就多一次 `read_pixels` 加一轮逐像素转换。

失败时的回滚语义

软件呈现失败会 `std::mem::take(&mut; damage_rollback_pending)`，若为真则清空损伤跟踪器，并无条件清空图层缓存。也就是说：只要本帧已经动过跟踪器状态而没能上屏，下一帧就退回全量重绘。这比试图精确回滚要简单得多，而且是唯一可证明正确的选择。

9 损伤跟踪：DamageTracker

`damage.rs` 是整个后端里逻辑密度最高的文件。它跨帧保存每层的快照，把两帧快照 diff 成损伤区域，再把损伤沿层级和 backdrop 依赖关系传播，最后简化成少量矩形。

9.1 数据结构

```
struct DamageTracker {
    snapshots: HashMap<RenderNodeId, LayerSnapshot>,
    last_damage: HashMap<RenderNodeId, DamageRegion>, // draw_layer 里子层重绘要查
}
struct LayerSnapshot { render_bounds, items: Vec<ItemSnapshot> }
struct ItemSnapshot { source: RenderNodeId, version: {content, placement}, bounds: Bounds, kind: ItemKind }
enum ItemKind { Draw,
    Layer { source, transform: Affine, program: ProgramFingerprint,
        backdrop_expansion: SampleExpansion } }
struct DamageRegion { rects: Vec<Bounds> } // 逻辑坐标，非物理像素
```

`ItemKind::Layer` 里带上 `ProgramFingerprint` 是关键：效果程序换了（比如模糊半径变了）即使内容版本没动，也必须判定为脏。`PlacementVersion{scene, dynamic}` 分离场景结构变化与动画等动态变化。

9.2 update：一次完整的损伤计算

```
fn update(frame) -> DamageRegion:
    effect_expansions = layer_effect_expansions(frame) // 每个子层被父放置时取到的最大视觉外扩

    for layer in frame.layers.iter().rev(): // 逆序 → 子层先算完，父层能读到子层的脏区
        next = layer_snapshot(frame, layer)
        dirty = match snapshots.get(&layer.source) {
            Some(prev) => diff_layer(prev, &next, &dirty_by_layer),
            None      => DamageRegion::full(layer.render_bounds), // 新层 → 整层脏
        }
        dirty.expand_sample(effect_expansions[&layer.source]) // 模糊/阴影会把脏区向外糊开
        dirty_by_layer.insert(layer.source, dirty)

    propagate_composite_prefix_damage(frame, &mut dirty_by_layer) // backdrop 依赖 + 父子传播
    for region in dirty_by_layer.values_mut() { region.simplify() }
    snapshots = next_snapshots; last_damage = dirty_by_layer
    return last_damage[root_layer.source]
```

9.3 diff_layer 的三种情况

条件	结果
<code>render_bounds</code> 变了	旧界 + 新界全脏，立即返回。
条目序列“稳定” (长度相同且逐项 <code>source</code> 与 <code>kind</code> 一致)	走 <code>diff_ordered_items</code> 逐项比对，只有真正变化的条目贡献脏区。
条目序列变了	找到第一个不一致的下标 <code>first_changed</code> ：之前的部分照常逐项 diff，之后的部分（新旧两边）全部条目的包围盒都算脏。

逐项比对 `diff_item` 的判定：

```
if old.placement == new.placement && old.bounds == new.bounds && old.kind == new.kind:
    match new.kind {
        Layer { source, transform, .. } =>
            if let Some(child) = child_dirty.get(source) { dirty.add_transformed(child, transform, new.bounds) }
            else if old.content != new.content { dirty.add(new.bounds) }
        Draw if old.content != new.content => dirty.add(new.bounds),
        Draw => {} // 完全没变
    }
else:
    dirty.add(old.bounds); dirty.add(new.bounds) // 位置/几何/种类变了 → 新旧都脏
```

关键之处：图层条目不是简单地“子层脏就整块脏”，而是把子层的脏区按合成变换映射到父层坐标再与实例包围盒求交（`add_transformed`）。一个滚动列表里改一行文字，父层只会脏那一行对应的矩形。

同时，`diff_ordered_items` 会在遍历中维护一个累积脏区，每遇到一个带 backdrop 的图层条目就问：“到目前为止累积的脏区，经过 backdrop 采样外扩之后，有没有落进这个实例的范围？”有就把这部分也算进去——因为背景变了，读背景的东西必须重画。

```
fn backdrop_damage(&self, output_bounds, expansion) -> Self:
    for rect in &self.rects:
        expanded = rect 各边按 expansion.{left,top,right,bottom} 外扩
        if let Some(affected) = expanded & output_bounds { damage.add(affected) }
```

9.4 propagate_composite_prefix_damage

两趟传播，处理 `diff_layer` 看不到的跨层依赖：

- 前缀链传播：对每个带 `destination_prefix` 的实例，沿 `parent` 指针把整条前缀链收集出来并反转（祖先在前）。对链上每个祖先，取它的脏区，逐级用 `through_prefix_placement` 变换到下一级坐标系（逆变换 + 父裁剪 + 采样外扩 + 子裁剪），最后用 `backdrop_damage` 投影到该实例的世界包围盒上。祖先层内容变了 → 读它当背景的后代要重画。
- 父子上卷：逆序遍历所有图层，把每个子层的脏区用 `add_transformed(dirty, composite.transform, world_bounds)` 累加到父层。这保证父层的重绘区域覆盖所有需要重新合成的子层位置。

9.5 simplify：控制矩形数量

损伤是靠 `append` 累积的：一帧里同一片像素可能经由前缀链、backdrop 外扩、父子传播三条路径各贡献一个几乎重合的矩形。每个矩形都要花一条 Skia `Region` band、一条裁剪 band，在软件路径上还要一次 `read_pixels` 加逐像素转换。

```
常量：MAX_RECTS = 32    MERGE_SLACK = 0.25    PAIRWISE_LIMIT = 64

fn simplify():
    if rects.len() < 2      { return }
    if rects.len() > PAIRWISE_LIMIT { return band_merge() } // O(n log n)

    for _ in 0..2 { merge_pass(); if 数量没变 { break } } // 两趟：一次合并可能开启新的合并机会
    while rects.len() > MAX_RECTS { if !merge_closest_pair() { break } }

fn merge_pass(): // 包含 → 丢弃 / 被包含 → 替换 / 相交且并集浪费 ≤ 25% → 合并
fn band_merge(): // 按 (min.y, min.x) 排序，切成 MAX_RECTS 组，每组取并集
fn merge_closest_pair() // 选并集浪费面积最小的一对合并；O(n²) 但 n 已被 PAIRWISE_LIMIT 限住
```

不变量：只增不减

这里的每一个操作都只会扩大覆盖面积，从不缩小。简化后的区域永远可以安全重绘——最多浪费一点工作，绝不会漏掉像素。丢一个像素就意味着屏幕上留下一块陈旧内容。测试 `simplify_never_drops_coverage` 在 2 / 17 / 64 / 65 / 200 五个规模上验证覆盖不丢失，`simplify_bounds_a_huge_region_without_losing_coverage` 用 20,000 个矩形验证分带路径在 100 ms 内完成（成对路径在这个规模上是 $O(n^3)$ ）。

10 缓存与资源池体系

后端里一共有七处跨帧或跨调用的复用结构，职责与失效条件各不相同：

名称	类型 / 容量	键	失效与驱逐
<code>layer_cache</code> <code>LayerSurfaceCache</code>	<code>HashMap</code> / 默认 128 MiB	<code>LayerCacheId</code>	命中还要求 <code>source</code> 与 <code>SurfaceGeometry</code> (bounds + 像素尺寸 + scale 位模式) 完全一致。帧末 <code>retain(live_layer_caches)</code> ，再按 <code>CachePolicy::Auto</code> 的 LRU 驱逐至预算内。

名称	类型 / 容量	键	失效与驱逐
<code>surface_pool</code> <code>SurfacePool</code>	<code>HashMap</code> 分档 默认 64 MiB	精确像素尺寸	闲置超 <code>MAX_POOL_IDLE_FRAMES = 60</code> 帧的尺寸档整档丢弃； <code>put</code> 超预算则不收。 <code>take</code> 会先清空表面（load-op / memset，远比重新分配便宜）。
<code>source_images</code>	moka <code>Cache</code> / 128 MiB	<code>ImageKey</code>	按权重（ <code>image_bytes</code> ）驱逐； <code>data_id</code> 不匹配即视为过期并重新上传。
<code>image_cache</code>	moka <code>Cache</code> / 128 MiB	<code>CachedImageKey</code>	键含 <code>data_id · transform · bytes</code> 。存的是应用了 <code>ImageTransform</code> （EXIF 旋转/翻转）之后的图像；恒等变换直接用源图，不进此缓存。
<code>vector_paths /</code> <code>vector_scenes /</code> <code>gradients</code>	<code>LocalLru</code> / 4096 · 1024 · 256	<code>PathDataId ·</code> <code>VectorSceneId ·</code> <code>GradientKey</code>	纯 LRU（ <code>VecDeque</code> 记录顺序 + <code>HashMap</code> 存值），无预算概念。
<code>text_blob_cache</code>	<code>HashMap</code>	<code>TextLayoutHandle</code>	两条失效路径：字体 <code>epoch</code> 变化则条目作废；每 120 帧修剪 120 帧未用的条目。另有 <code>handle_node_lifecycle(Removed(owner))</code> 按所属节点即时清除。
<code>font_cache</code>	<code>HashMap</code>	(<code>FontId</code> , <code>FontWeight</code>)	<code>backend.epoch()</code> 一变就整表清空。
<code>runtime_effects</code>	<code>HashMap</code>	效果名	值是 <code>RuntimeEffect</code> 。永不失效——SkSL 源码是编译期常量。
<code>font_mgr</code>	<code>Option</code>	—	首次使用时构建，之后永久复用。 <code>FontMgr::new()</code> 在 macOS 上是一次 CoreText 枚举，约 30 ms；曾经每次 <code>typeface</code> 缓存未命中都构建一个，十种字体就是首帧 300 ms。

10.1 SurfacePool 的工作方式

```

struct PooledSurface { surface: Surface, last_used: u64 }
struct SurfacePool { by_extent: HashMap<(u32,u32), Vec<PooledSurface>>, bytes: u64, frame: u64 }

begin_frame(): frame += 1 ; 丢弃 last_used + MAX_POOL_IDLE_FRAMES < frame 的整档，相应扣减 bytes
take(w,h):      弹出一块 → 清空画布状态与像素 → 返回；未命中返回 None（调用方新分配）
put(surface,budget): bytes + surface_bytes(s) <= budget 才收下；否则直接丢弃
clear():        整池清空

```

为什么值得：一个普通的 effects 列表每帧要跑八个 filter pass，每个都分配一块表面、用一帧、丢掉，而且尺寸帧帧相同（场景几何没变）。Skia 的资源缓存会复用底层分配，但每次请求仍要在其上构建一个 surface、一个 render target 和一个 device。池化把这三层也省了。

11 文本与字体流程

文本被拆成两半：`SkiaTextBackend`（`text.rs`）负责排版与字形光栅化，由上层的 `TextHost<T>` 持有；`SkiaBackend`（`backend.rs`）负责把排版结果画到画布上。后端对 `T: TextBackend` 是泛型的——默认是 `SkiaTextBackend`，而 `xui-winit` 实际装配的是 `xui-text-engine` 的 `cosmic-text` 实现。

11.1 SkiaTextBackend 实现的三个 trait

Trait	方法与行为
FontDatabase	<code>epoch()</code> 单调计数，任何字体集合或 <code>scale</code> 变化都自增（下游据此整表失效）； <code>load_system_fonts()</code> 重建 <code>FontMgr</code> ； <code>load_font_bytes(Arc<[u8]>)</code> 注册进 <code>TypefaceFontProvider</code> ； <code>query(&FontQuery)</code> 先查自定义再查系统； <code>font_data(id)</code> 返回 <code>FontDataRef</code> 。
Shaper	<code>layout_paragraph(state, input)</code> 构建 <code>SkParagraphStyle / SkTextStyle</code> （字号、字重、字族、字距、行高比例并 <code>set_height_override(true)</code> ），经 <code>ParagraphBuilder</code> 建段落按约束布局： <code>Definate(w)</code> 直接用 <code>w</code> ； <code>Unbound</code> 先按 1,000,000 排一次再按 <code>max_intrinsic_width</code> 重排； <code>MinSize</code> 同理用 <code>min_intrinsic_width</code> 。最后 <code>extract_layout</code> 抽出行/run/glyph/cluster 四组数据，段落本身留在 <code>SkiaParagraphState</code> 里。
GlyphRasterizer	<code>rasterize(SkiaGlyphKey)</code> ：按 <code>font_size × scale</code> 建 <code>Font</code> ，开 <code>subpixel</code> 与 <code>SubpixelAntiAlias</code> ；取字形包围盒各边外扩 1 px；画进一块光栅表面并 <code>read_pixels</code> ；检测是否为彩色字形（存在 <code>R≠G</code> 或 <code>G≠B</code> 的非透明像素），非彩色则把 RGB 全部填成 alpha，返回 <code>Mask</code> ，否则返回 <code>Color</code> 。

11.2 绘制通路：TextBlob 而非逐字形位图

值得强调的一点：`draw_text` 并不使用上面的 `GlyphRasterizer`。它拿排版结果里的 run，为每个 run 解析出 `SkTypeface`，用 `TextBlobBuilder::alloc_run_pos` 直接填字形 ID 与位置，交给 Skia 自己的字形缓存去光栅化。测试 `draws_shaper_output_without_backend_rasterization` 明确锁定了这个行为。

```
draw_text(canvas, primitive, transform, opacity, text):
    handle = text.active_slot(node_id, slot)?           // 没有 ⇒ InvalidFrame("no active layout")
    layout = text.layout(handle)?                       // 没有 ⇒ InvalidFrame("layout is not resident")
    canvas.save() ; canvas.concat(transform) ; canvas.clip_rect(primitive.bounds)
    y_offset = match vertical_align { Top|Baseline => 0, Middle => (h - layout.h)/2, Bottom => h - layout.h }
    origin = (bounds.x, bounds.y + y_offset)

    1.选区：query.selection_rects(range) 逐矩形填色（关闭抗锯齿，避免相邻矩形接缝）
    2.正文：text_blob_for_layout(...) → canvas.draw_text_blob(blob, origin, solid_paint(color))
    3.光标：query.caret_rect(char_index)，失败则回退到行尾 + font_size*1.2 高度，画一条竖线
    4.IME：对 ime.range 的每个矩形在底部画下划线
    5.draw_text_decorations(...)：下划线/删除线等装饰
    canvas.restore_to_count(save)

text_blob_for_layout(backend, handle, owner, layout):
    命中且 cached.font_epoch == backend.epoch() ⇒ 刷新 last_used_frame 并返回克隆
    否则 build_text_blob() 后插入缓存（记录 owner，供 handle_node_lifecycle 定向清除）

build_text_blob(backend, layout):
    for run in layout.runs:
        glyphs = layout.glyphs[run.glyph_range]        // 越界 ⇒ InvalidFrame
        typeface = typeface_for_font(backend, run.font_id, run.font_weight)?
        font = Font::from_typeface(typeface, run.font_size.max(1.0)) ; subpixel + SubpixelAntiAlias
        let (ids, positions) = builder.alloc_run_pos(&font, glyphs.len(), None)
        逐字形填入 GlyphId (u32~u16 失败即 InvalidFrame) 与 draw_pos
    has_glyphs.then(|| builder.make()).flatten()
```


11.3 typeface_for_font : 三种字体来源

```
epoch 变化 => font_cache.clear()          缓存键 = (FontId, FontWeight)

match backend.font_data(font_id)? {
  Bytes { bytes, index } => font_mgr().new_from_data(bytes, index)
  SystemMemory { bytes, index, family, postscript_name, style, stretch, .. }
    => system_typeface(family, ps_name, sk_font_style(weight, stretch, style))
    .or_else(|| load_font_from_bytes(bytes, index))
  System { path, index, family, postscript_name, style, stretch, .. }
    => system_typeface(...)
    .or_else(|| load_font_from_path(path, index)) // mmap2 内存映射
}
// system_typeface: 先按 family 枚举全部 style, 用 post_script_name 精确匹配;
// 匹配不到再退回 font_mgr().match_family_style(family, style)
```

12 平台呈现后端

`WindowPresenter::new(window)` 是运行时选择点：先看 `XUI_SKIA_GPU` (`0/off/false` 强制软件)，再试当前平台的 GPU 呈现器；失败则打印一行警告并回落 `softbuffer`——虚拟机、远程会话和无可驱动驱动的 CI 容器都属于这种常见情形。

```
enum WindowPresenter {
  #[cfg(macos)] Metal(MetalPresenter),          #[cfg(windows)] Direct3D(Direct3DPresenter),
  #[cfg(linux)] Vulkan(Box<VulkanPresenter>),    Software(SoftwarePresenter),
}
统一接口：new() -> (Self, Option<DirectContext>) · resize(ctx,w,h) · acquire_surface(ctx,w,h) -> Surface
present(ctx, surface) · software_mut() -> Option<&mut SoftwarePresenter>
```

12.1 Metal (macOS) `metal.rs`

初始化	从 AppKit <code>ns_view</code> 取出 <code>CAMetalLayer</code> ；设 <code>device</code> 、 <code>MTLPixelFormat::BGRA8Unorm</code> 、 <code>framebufferOnly = false</code> 、 <code>presentsWithTransaction = false</code> 、初始 <code>drawableSize</code> ；建 <code>MTLCommandQueue</code> ； <code>gpu::direct_contexts::make_metal(&backend, None)</code>
每帧	<code>nextDrawable() -> mtl::TextureInfo -> backend_render_targets::make_mtl -> wrap_backend_render_target(BGRA8888, sRGB) -> 绘制 -> flush_and_submit_surface -> drop(surface) -> commandBuffer.presentDrawable(drawable) + commit()</code>
要点	<code>resize</code> 只需 <code>setDrawableSize</code> 。每帧都是全新 <code>drawable</code> ，没有任何需要提前释放的缓存引用，所以这条路径的 <code>resize</code> 忽略传入的 <code>context</code> 。

12.2 Direct3D 12 (Windows) `d3d.rs`

初始化	枚举 DXGI 适配器并跳过 WARP 软件适配器（宁可用 <code>softbuffer</code> 兜底，也不假装 GPU 加速）； <code>D3D12CreateDevice(FEATURE_LEVEL_11_0)</code> ；DIRECT 命令队列； <code>DXGI_SWAP_EFFECT_FLIP_DISCARD</code> 交换链， <code>BUFFER_COUNT = 3</code> 、 <code>DXGI_FORMAT_B8G8R8A8_UNORM</code> ； <code>ID3D12Fence</code> + 事件句柄； <code>gpu::direct_contexts::make_d3d</code>
每帧	<code>GetCurrentBackBufferIndex() -> 等待该缓冲记录的 fence 值 -> GetBuffer(index) -> 以 D3D12_RESOURCE_STATE_PRESENT 声明并 wrap -> 绘制 -> flush_surface_with_access(Present) + submit(None) -> drop(surface) -> swapchain.Present(1, 0) -> queue.Signal(fence, ++fence_value) 并记入 buffer_fences[index]</code>
帧节奏	单 fence：每次 Present 之后把新的 fence 值记在该后台缓冲上，重新取到它时才等。这样最多有 <code>BUFFER_COUNT</code> 帧在飞，而不是像“每帧 Present 后 <code>WaitForSingleObject</code> ”那样每帧一次全停。直接队列按提交顺序执行，所以此刻排入的 Signal 必然在本帧渲染完成之后才到达。
resize 的坑	<code>ResizeBuffers</code> 会重建每一个后台缓冲，只要还有任何一个引用未释放就返回 <code>DXGI_ERROR_INVALID_CALL</code> 。仅仅丢掉 Skia surface 是不够的——被包装的 render target 会留在上下文的资源缓存里继续持有 <code>ID3D12Resource</code> 。所以必须先 <code>flush_submit_and_sync_cpu()</code> 再 <code>free_gpu_resources()</code> ，然后等 fence 到最新值，最后才 <code>ResizeBuffers</code> 。

12.3 Vulkan (Linux, X11 与 Wayland) `vulkan.rs`

初始化	<code>VkInstance</code> → 物理设备筛选（必须支持 <code>VK_KHR_swapchain</code> ）→ <code>VkDevice</code> → 表面格式与呈现模式选择 → 交换链， <code>PREFERRED_IMAGE_COUNT = 3</code>
每帧	<code>acquireNextImage</code> （带 fence）→ 等 fence → wrap → 绘制 → <code>submit(SyncCpu::Yes)</code> → <code>vkQueuePresentKHR</code> （不带等待信号量）
已知取舍	skia-safe 0.99 没有暴露构造 <code>BackendSemaphore</code> 的途径，也无法往 <code>GrFlushInfo</code> 里塞信号量（ <code>gpu::FlushInfo</code> 的字段是私有的，crate 里带着一条相关 TODO）。于是这个呈现器改在 CPU 上同步：用 fence 保证呈现引擎确实用完了该图像，用 <code>SyncCpu::Yes</code> 保证渲染确实完成。正确，但每帧两次 CPU/GPU 往返；等上游 API 就位后可以原地换成信号量。

12.4 softbuffer (全平台兜底) `present.rs`

初始化	<code>Context::new(window) + SoftSurface::new(&context, window)</code>
每帧	不参与 <code>acquire_surface / present</code> ；由 <code>end_frame</code> 直接操作 <code>presenter.surface</code> （见 §8.2）
要点	这条路径上 <code>acquire_surface</code> 与 <code>present</code> 一律返回 <code>InvalidFrame</code> ——它们不该被调用，因为 <code>gpu_context</code> 是 <code>None</code> ，两处调用点都由 <code>if let (Some(presenter), Some(context))</code> 守住。

13 效果实现与 SkSL

四段 SkSL 常量嵌在 `backend.rs` 里，编译后按名字缓存在 `runtime_effects`：

常量	类型	uniform	作用
<code>PIXELATE_SKSL</code>	shader	<code>float2 block</code>	把采样坐标量化到 block 网格，产生马赛克
<code>REFRACTION_SKSL</code>	shader	<code>float2 center · float2 amount</code>	以 center 为中心做径向位移折射， <code>amount.y</code> 控制通道分离量
<code>CHROMATIC_ABERRATION_SKSL</code>	shader	<code>float2 offset</code>	R/B 通道按 <code>±offset</code> 偏移采样，G 通道不动
<code>COMPOSITE_SKSL</code>	blender	<code>int blend · int op</code>	Skia 内建 BlendMode 表达不了的“混合模式 × 合成算子”组合，在着色器里按索引选分支

```
runtime_effect(name, source, blender) -> RuntimeEffect    // 命中 runtime_effects 直接返回
  make_for_blender(source) | make_for_shader(source)      // 失败 ⇒ RuntimeEffect{effect,message}

runtime_filter(name, source, uniforms) -> ImageFilter
  builder = RuntimeShaderBuilder::new(effect)
  for (u,v) in uniforms { builder.set_uniform_float(u,v)? } // 失败 ⇒ RuntimeUniform{..}
  image_filters::runtime_shader(&builder, "source", None).ok_or(RuntimeShader(name))
```

所有以像素为单位的参数（ σ 、块尺寸、偏移、半径）在进 Skia 前都会除以 `scale_factor` 换回逻辑单位——画布已经带了 scale 变换，不除就会在高 DPI 上被放大两次。

13.1 其它绘制细节

- 裁剪链：`apply_clip_chain` 递归而非迭代——链是子→父存储的，要先裁外层。递归深度等于裁剪节点的嵌套深度（很浅），而原先的迭代写法每次调用都要分配一个 `Vec`，且这个函数每绘制一项就跑一次。
- 裁剪形状：`apply_clip_shape` 在变换保持轴对齐时把矩形/圆角矩形按原样下发，让 Skia 走 `scissor` 或 `rrect` 快路径；旋转、斜切和路径裁剪才退化成路径形式（需要光栅化裁剪掩码）。
- 渐变：`GradientKey([u32;18])` 把决定一个渐变的每一个输入都编码进键。注释点明了设计意图：字段是解构出来的而不是通过访问器读的，这样往 `ComputedColorStyle` 加字段时会在这里编译失败，而不是悄悄生成两个不同渐变共享的键。

- 矢量场景：`compiled_vector_scene` 把 `VectorCommand` 预编译成 `CompiledVectorCommand`（内含已构建的 Skia Path），按 `VectorSceneId` 缓存为 `Arc<_>`；单条路径另有 `vector_paths` 按 `PathDataId` 缓存。
- 空图像：`transparent_image` 返回一个 1×1 全透明像素并按需拉伸。空掩码和 `backdrop-only` 通路的层内容占位符原先各要分配一块整层大小的表面、清空、快照，只为表达“什么都没有”。

14 错误语义、优化开关与统计

14.1 SkiaBackendError

分类	变体
表面与像素	<code>SurfaceAllocation{w,h}</code> · <code>PixelRead</code>
帧契约	<code>InvalidFrame(String)</code> （缺图层/实例/裁剪链、前缀不匹配、阶段顺序错误、字形 ID 溢出、文本布局缺失……全部归到这里）
渲染计划	<code>RenderPlan([from] PlanError)</code> · <code>MissingResource(usize)</code> · <code>MissingMaskImage(ImageKey)</code>
SkSL	<code>RuntimeEffect{effect,message}</code> · <code>RuntimeShader(name)</code> · <code>RuntimeUniform{effect,message}</code>
平台	<code>SoftBuffer([from])</code> · <code>MetalInitialization/MetalPresentation</code> · <code>VulkanInitialization/VulkanPresentation</code> · <code>Direct3DInitialization/Direct3DPresentation</code> · <code>NoGpuPresenter</code>
字体	<code>FontDataError(String)</code>

统一的恢复策略是丢弃全部增量状态：绘制失败清 `damage_tracker` / `layer_cache` / `surface_pool`；呈现失败按 `damage_rollback_pending` 决定是否清跟踪器，并无条件清图层缓存。理由在代码注释里写得很直白——目标表面已经被写了一半，唯一可靠的恢复就是下一帧全量重绘。

14.2 SkiaOptimizations

```
pub struct SkiaOptimizations { pub cull: bool, pub surface_pool: bool }
impl Default { cull: true, surface_pool: true } // 生产路径
pub const NONE: Self = { cull: false, surface_pool: false } // 参考路径（全部快路径关闭）
```

这两个开关只为测试而存在。文档注释说明了理由：让测试把同一个场景渲染两遍再比对像素，是保持快路径诚实的唯一廉价手段。`SkiaBackendOptions` 另有 `clear_color`、`layer_cache_budget_bytes`（128 MiB）与 `surface_pool_budget_bytes`（64 MiB）。

14.3 SkiaFrameStats（每帧重置）

组	字段
损伤	<code>frame_index</code> · <code>root_damage_rects</code> · <code>root_damage_area_sum</code>
绘制量	<code>primitive_draws</code> · <code>layer_draws</code> · <code>image_snapshots</code>
计划	<code>render_plans</code> · <code>render_passes</code> · <code>planned_transient_resources</code> · <code>planned_transient_slots</code> · <code>planned_transient_texels</code> · <code>planned_peak_live_texels</code>
表面	<code>transient_surface_allocations</code> · <code>transient_surface_reuses</code> · <code>offscreen_surface_allocations</code> · <code>pooled_surface_reuses</code>
backdrop	<code>backdrop_materializations</code> · <code>backdrop_materializations_avoided</code>
剔除	<code>items_culled</code> · <code>layer_instances_culled</code> （后者每省一个还连带省掉一块离屏表面、一次子树遍历、一次快照和一个合成 pass）

另有 `SkiaLayerCacheStats` (`layer_cache_stats()`，累计而非逐帧)：`hits` · `misses` · `entries` · `resident_bytes` · `partial_updates` · `full_updates` · `dirty_regions`。

15 测试与调试

15.1 单元测试 (`backend.rs` / `damage.rs` 内嵌)

模块	覆盖点
<code>text_draw_tests</code>	<code>draws_shaper_output_without_backend_rasterization</code> ——锁定“走 <code>TextBlob</code> 而非后端字形位图”
<code>blit_tests</code>	<code>blit_replaces_the_destination_one_to_one</code> (Src 语义) · <code>blit_ignores_the_canvas_transform</code> (画布遗留变换不影响拷贝)
<code>surface_pool_tests</code>	<code>a_recycled_surface_carries_no_canvas_state</code> · <code>the_pool_drops_extents_that_go_unused</code> · <code>the_pool_stops_at_its_budget</code>
<code>damage::tests</code>	<code>transformed_damage_is_clipped</code> · <code>simplify_folds_a_contained_rect_away</code> · <code>simplify_merges_overlapping_rects_it_barely_grows</code> · <code>simplify_keeps_rects_whose_union_would_waste_space</code> · <code>simplify_never_drops_coverage</code> · <code>simplify_bounds_a_huge_region_without_losing_coverage</code> · <code>backdrop_expands_only_intersecting_damage</code>

15.2 集成测试

文件	作用 (均在 <code>xui-skia/tests/</code> 下)
<code>optimization_parity.rs</code> (423 行)	把同一场景渲染两遍比像素： ① <code>SkiaOptimizations::default()</code> vs <code>NONE</code> ——抓住画错东西的快路径； ② 增量重绘的后端 vs 每帧 <code>invalidate()</code> 的后端——抓住覆盖不全的损伤，也就是所有剔除赖以成立的那个假设。 场景刻意覆盖全部分支：可内联的嵌套容器、圆角裁剪、变换子树、模糊层、backdrop 层、渐变、文本，以及行数远超视口的滚动容器（可剔除）。
<code>raster_budget.rs</code> (151 行)	带真实文本整形与真实光栅化的整帧成本测量，行数取 25/100/400/1600 四档，视口 900×700。补足 <code>xui</code> 的 <code>frame_bench</code> （后者止步于已构建的帧）。 文件头注明了口径：无窗口 Skia 在 CPU 上光栅化，而 macOS 出货路径跑 Metal，所以这不是设备端光栅成本的预测；但两条路径现在都遵守损伤跟踪，所以“空闲”与“动画中”两列之间的差距对二者是同一份收益。

```
cargo test -p xui-skia --test optimization_parity
cargo test -p xui-skia --release --test raster_budget -- --ignored --nocapture
```

15.3 环境变量

变量	效果
<code>XUI_DEBUG_FRAME=1</code>	每帧打印一行损伤摘要： <code>frame_size_px</code> 、根层 <code>render_bounds</code> 、条目数、损伤矩形数、损伤面积和、是否 GPU。通过 <code>LazyLock</code> 只读一次——查环境变量要取进程级锁并分配内存，不能每帧做。
<code>XUI_SKIA_GPU=0 off false</code>	跳过 GPU 呈现器初始化，强制走 <code>softbuffer</code> 软件路径。

16 代码位置索引

按调用顺序排列的主要函数与其所在行（基于分析时的 `ani` 分支）：

阶段	函数	位置
入口	<code>App::render</code>	<code>xui/src/app.rs:224</code>

阶段	函数	位置
	<code>RenderBackend</code> trait 定义	<code>xui/src/render/backend.rs:4</code>
构造	<code>SkiaBackend::new</code> / <code>new_with_options</code> / <code>headless</code>	<code>backend.rs:368</code> / <code>372</code> / <code>410</code>
	<code>WindowPresenter::new</code>	<code>present.rs:59</code>
begin	<code>begin_frame</code>	<code>backend.rs:2165</code>
	<code>ensure_surface</code>	<code>backend.rs:496</code>
	<code>SurfacePool::begin_frame</code> / <code>take</code> / <code>put</code>	<code>backend.rs:231</code> / <code>251</code> / <code>270</code>
submit	<code>submit</code>	<code>backend.rs:2193</code>
	<code>DamageTracker::update</code>	<code>damage.rs:271</code>
	<code>diff_layer</code> / <code>diff_ordered_items</code> / <code>diff_item</code>	<code>damage.rs:363</code> / <code>410</code> / <code>429</code>
	<code>propagate_composite_prefix_damage</code>	<code>damage.rs:485</code>
	<code>DamageRegion::simplify</code> / <code>band_merge</code> / <code>merge_pass</code>	<code>damage.rs:82</code> / <code>113</code> / <code>128</code>
	<code>analyze_frame</code> / <code>prepare_image</code>	<code>backend.rs:600</code> / <code>650</code>
	<code>draw_frame</code> → <code>redraw_layer_region</code> → <code>draw_layer</code>	<code>backend.rs:674</code> / <code>701</code> / <code>744</code>
	<code>execute_backdrop_only</code>	<code>backend.rs:948</code>
	<code>execute_instance</code> → <code>execute_plan</code>	<code>backend.rs:987</code> / <code>1029</code>
	<code>execute_filter_pass</code> / <code>execute_composite_pass</code>	<code>backend.rs:1111</code> / <code>1249</code>
	<code>resolve_resource</code> / <code>resolve_mask_image</code>	<code>backend.rs:1354</code> / <code>1383</code>
	<code>blit_to_swapchain</code> / <code>blit_surface</code>	<code>backend.rs:1428</code> / <code>2633</code>
	<code>LayerSurfaceCache::acquire</code> / <code>release</code> / <code>finish_frame</code>	<code>cache.rs:97</code> / <code>125</code> / <code>155</code>
end	<code>end_frame</code>	<code>backend.rs:2261</code>
	<code>physical_damage_rects</code> / <code>copy_surface_damage</code>	<code>backend.rs:2543</code> / <code>2581</code>
	<code>MetalPresenter::acquire_surface</code> / <code>present</code>	<code>metal.rs:86</code> / <code>117</code>
	<code>Direct3DPresenter::acquire_surface</code> / <code>present</code> / <code>resize</code>	<code>d3d.rs:203</code> / <code>246</code> / <code>163</code>
	<code>VulkanPresenter::acquire_surface</code> / <code>present</code> / <code>recreate_swapchain</code>	<code>vulkan.rs:218</code> / <code>279</code> / <code>356</code>
图元	<code>draw_shape</code> / <code>draw_vector</code> / <code>draw_image</code> / <code>draw_text</code>	<code>backend.rs:3102</code> / <code>3189</code> / <code>1816</code> / <code>1875</code>
	<code>apply_clip_chain</code> / <code>apply_clip_shape</code>	<code>backend.rs:1753</code> / <code>2752</code>
	<code>gradient_shader</code> / <code>GradientKey::new</code>	<code>backend.rs:3036</code> / <code>2978</code>
文本	<code>SkiaTextBackend::layout_paragraph</code> / <code>extract_layout</code> / <code>rasterize</code>	<code>text.rs:177</code> / <code>250</code> / <code>414</code>
	<code>typeface_for_font</code> / <code>build_text_blob</code> / <code>text_blob_for_layout</code>	<code>backend.rs:2006</code> / <code>2079</code> / <code>2119</code>
SkSL	<code>PIXELATE</code> / <code>REFRACTION</code> / <code>CHROMATIC_ABERRATION</code> / <code>COMPOSITE</code>	<code>backend.rs:2372</code> / <code>2382</code> / <code>2400</code> / <code>2412</code>

阅读建议

第一次读 `backend.rs` 时，按 §16 的 “② submit” 那一段的顺序自上而下走一遍，先不看 `backdrop` 与 `passthrough` 分支（它们是少数派路径），把 `draw_layer` 的 `Draw` 分支和不需要 `backdrop` 的 `Layer` 分支跑通，整条主干就清楚了。